

Reviewer Pack — Minimal Diophantine Complexity Correlation with Resolution P...

Entry 4c98a57f0315 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 13:54:07 UTC

Minimal Diophantine Complexity Correlation with Resolution Proof Width

Entry ID: 4c98a57f0315 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 13:54:07 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (Diophantine Equations) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Tseitin formula φ , the minimal number of solutions to a system of Diophantine equations that can be satisfied by all satisfying assignments of φ is linearly correlated with its resolution proof width $w(\varphi)$, such that $|S(\varphi)| = \Theta(w(\varphi))$, where $S(\varphi)$ denotes the set of Diophantine equations.

Rationale (proposer's reasoning):

Diophantine equations are a fundamental tool in arithmetic geometry, and their solution sets can be used to encode properties of Boolean functions. This conjecture posits a direct link between the complexity of solving these equations and the resolution proof complexity, which could provide insights into the inherent difficulty of problems in computational complexity theory.

Taxonomy category: ArithmeticGeometryToResolutionProof (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 32a25eed9fb59531

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For Tseitin formulas φ , if $|S(\varphi)|$ is within a factor of 2 from $\Theta(w(\varphi))$, where $|S(\varphi)|$ is the number of Diophantine equations and $w(\varphi)$ is the resolution proof width, then the conjecture is supported. If any $|S(\varphi)|/w(\varphi)$ ratio deviates by more than a factor of 2 from 1 for at least one seed, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Diophantine Equations' AND 'resolution proof complexity' - 'Minimal Diophantine Complexity' AND 'Tseitin formula resolution width correlation' - 'Arithmetic Geometry' AND 'correlation with resolution proof width for Tseitin formulas'

Top relevant hits considered: - [http://arxiv.org/abs/1609.06986v1] More on Diophantine sextuples - [http://arxiv.org/abs/1310.8140v2] Primes Solutions Of Linear Diophantine Equations - [http://arxiv.org/abs/2306.00922v3] Sur la Resolution des Equations de Troisième et Quatrième Degré - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/1603.03800v3] Diophantine approximation on matrices and Lie groups - [http://arxiv.org/abs/astro-ph/0407622v3] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly's Eye - [http://arxiv.org/abs/0706.3841v1] Arithmetic lattices and weak spectral geometry - [http://arxiv.org/abs/2603.03026v1] Any Resolution Any Geometry: From Multi-View To Multi-Patch

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random

def generate_tseitin_formula(n: int, m: int) -> tuple:
    variables = list(range(1, n + 1))
    literals = [f"v{i}" for i in variables] + [f"~v{i}" for i in variables]
    clauses = []

    # Generate clauses
    for _ in range(m):
        clause = random.sample(literals, 2)
        clauses.append(clause)

    return variables, literals, clauses

def solve_diophantine_equations(variables: list, literals: list, clauses: list) -> int:
    # Placeholder function to simulate solving Diophantine equations
    # In practice, you would implement a proper algorithm here
    return len(literals)

def compute_resolution_proof_width(clauses: list) -> int:
```

```

# Placeholder function to simulate computing resolution proof width
# In practice, you would implement a proper algorithm here
return len(clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    m = 10 * n

    variables, literals, clauses = generate_tseitin_formula(n, m)
    num_solutions = solve_diophantine_equations(variables, literals, clauses)
    proof_width = compute_resolution_proof_width(clauses)

    metric_value = abs(num_solutions - proof_width) / max(1, proof_width)
    conjecture_holds = 0.5 <= metric_value <= 2.0
    counterexample = "" if conjecture_holds else f"num_solutions={num_solutions}, proof_width={proof_width}"

    return {
        "metric_name": "Diophantine Complexity / Resolution Proof Width Ratio",
        "metric_value": metric_value,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = (sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results)) ** 0.5
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ue': 0.8, 'instances_tested': 1, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
```

TRIAL: {'metric_name': 'Diophantine Complexity / Resolution Proof Width Ratio', 'metric_value': 0.8},
 TRIAL: {'metric_name': 'Diophantine Complexity / Resolution Proof Width Ratio', 'metric_value': 0.8},
 TRIAL: {'metric_name': 'Diophantine Complexity / Resolution Proof Width Ratio', 'metric_value': 0.8},
 TRIAL: {'metric_name': 'Diophantine Complexity / Resolution Proof Width Ratio', 'metric_value': 0.8},
 TRIAL: {'metric_name': 'Diophantine Complexity / Resolution Proof Width Ratio', 'metric_value': 0.8},
 TRIAL: {'metric_name': 'Diophantine Complexity / Resolution Proof Width Ratio', 'metric_value': 0.8},
 TRIAL: {'metric_name': 'Diophantine Complexity / Resolution Proof Width Ratio', 'metric_value': 0.8},
 TRIAL: {'metric_name': 'Diophantine Complexity / Resolution Proof Width Ratio', 'metric_value': 0.8},
 TRIAL: {'metric_name': 'Diophantine Complexity / Resolution Proof Width Ratio', 'metric_value': 0.8},
 TRIAL: {'metric_name': 'Diophantine Complexity / Resolution Proof Width Ratio', 'metric_value': 0.8},
 TRIAL: {'metric_name': 'Diophantine Complexity / Resolution Proof Width Ratio', 'metric_value': 0.8},
 RESULT: SUPPORTED mean=0.8 std=0.0 support_fraction=1.0

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses placeholder functions for solving Diophantine equations and computing resolution proof width, which do not implement the actual algorithms. The metric value is computed using these placeholders, which may not accurately reflect the true relationship between Diophantine complexity and resolution proof width.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code uses placeholder functions for solving Diophantine equations and computing resolution proof width, which may not accurately reflect the true relationship between these metrics. The critic's challenge indicates that the pre-registered support condition is not unambiguously met. | next: Implement actual algorithms for solving Diophantine equations and computing resolution proof width to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14467
2	preregistration	ollama_remote	glm4:latest	0	0	10149
3	novelty	ollama_remote	glm4:latest	0	0	8355
4	novelty	ollama_remote	glm4:latest	0	0	9904
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24413
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10733
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14659
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9733
9	critic	ollama_remote	glm4:latest	0	0	16508
10	judge	ollama_remote	glm4:latest	0	0	9172

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 128094 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/4c98a57f0315.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4c98a57f0315.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4c98a57f0315.tar.gz` (if generated)